

## 1. ServletContext Interface

### What is ServletContext?

ServletContext is an interface in the **Java Servlet API** that represents the **entire web application**.

It provides a way for servlets to:

- Communicate with the container
- Share data across the whole application

### Scope of ServletContext

- **Application-wide scope**
- Shared among **all servlets and JSPs**
- Created **once** when the application starts
- Destroyed when the application stops

That means:

- One ServletContext object per web application

### Common Uses

- Store global data
- Read configuration parameters
- Access server resources (files, paths)
- Logging

### Example: Setting and Getting Attributes

```
ServletContext context = getServletContext();
```

```
// Setting attribute
```

```
context.setAttribute("username", "admin");
```

```
// Getting attribute
```

```
String user = (String) context.getAttribute("username");
```

### Example: web.xml Context Parameter

```
<context-param>
```

```
  <param-name>appName</param-name>
```

```
<param-value>MyWebApp</param-value>
</context-param>
```

### Accessing it in Servlet:

```
ServletContext context = getServletContext();
String appName = context.getInitParameter("appName");
```

### Important Methods of ServletContext

| Method                                | Description            |
|---------------------------------------|------------------------|
| getAttribute(String name)             | Gets attribute         |
| setAttribute(String name, Object obj) | Sets attribute         |
| removeAttribute(String name)          | Removes attribute      |
| getInitParameter(String name)         | Gets context parameter |
| getRealPath(String path)              | Gets real file path    |
| log(String msg)                       | Writes log message     |

### Key Points

- Shared across entire application
- Used for **global data sharing**
- Different from ServletConfig (which is per servlet)

## 2. Web Application Listener

### What is a Listener?

A **listener** is a class that **listens to events** in a web application, such as:

- Application start/stop
- Session creation/destruction
- Request lifecycle

### Why use Listeners?

- Perform tasks automatically when events occur
- Initialize resources (like DB connections)

- Cleanup resources when application stops

## Common Listener Interfaces

| Listener               | Purpose               |
|------------------------|-----------------------|
| ServletContextListener | Application lifecycle |
| HttpSessionListener    | Session lifecycle     |
| ServletRequestListener | Request lifecycle     |

### 1. ServletContextListener

#### Methods

```
public void contextInitialized(ServletContextEvent sce)
public void contextDestroyed(ServletContextEvent sce)
```

#### Example

```
import javax.servlet.*;

public class AppListener implements ServletContextListener {

    public void contextInitialized(ServletContextEvent sce) {
        System.out.println("Application Started");
    }

    public void contextDestroyed(ServletContextEvent sce) {
        System.out.println("Application Stopped");
    }
}
```

#### Configuration (web.xml)

```
<listener>
  <listener-class>com.example.AppListener</listener-class>
</listener>
```

### 2. HttpSessionListener

```
public void sessionCreated(HttpSessionEvent se)
public void sessionDestroyed(HttpSessionEvent se)
```

👉 Used for:

- Tracking active users
- Session timeout handling

### **3. ServletRequestListener**

```
public void requestInitialized(ServletRequestEvent sre)  
public void requestDestroyed(ServletRequestEvent sre)
```

👉 Used for:

- Logging requests
- Monitoring request lifecycle